

Class 6: R Functions Lab

Mason Dinh (PID: A19140455)

This week we are introducing **R functions** and how to write our own R functions.

Questions to answer:

Q1. Write your first R function: add()

```
# Version A
# This function takes two numbers defined as x and y, and adds them together.

add <- function(x, y) {x + y}
add(4, 7)
```

```
[1] 11
```

```
# Version B
# Using sum() allows a single vector or two numbers.
# It takes the numbers within a vector and adds them up.
add <- function(x, y = 0) {sum(x, y)}
add(c(4, 7, 10))
```

```
[1] 21
```

```
# Version C
# The ... allows any number of arguments to be passed to sum()
add <- function(...) {sum(...)}
add(1, 2, 3, -4)
```

```
[1] 2
```

Q2. Write a generate_dna() function

```
# Version A
# This function randomly selects 'n' bases from the set of DNA nucleotides
# (A, T, G, C). The 'replace = TRUE' allows replacement so that a
# base can be picked more than once.
generate_dna <- function(n) {sample(c("A","T","G","C"), n, replace = TRUE)}
generate_dna(6)
```

```
[1] "G" "T" "C" "A" "A" "G"
```

```
# Version B
# This function generates a random vector of n DNA bases with replacement.
# The function 'If as_string = TRUE', combines into one string
# otherwise, return the vector.
generate_dna <- function(n, as_string = FALSE) {
  dna <- sample(c("A","T","G","C"), n, replace = TRUE)
  if (as_string) paste0(dna, collapse = "") else dna}
generate_dna(7, as_string = TRUE)
```

```
[1] "TTTAAAA"
```

```
# Version C
# This function generates a random DNA sequence and collapses into a single
# string. The 'cat()' function prints the result in FASTA format
# with >len(n) on the first line, followed by the sequence on the second.

generate_dna <- function(n) {
  dna <- paste0(sample(c("A","T","G","C"), n, replace = TRUE), collapse = "")
  cat(">len", n, "\n", dna, "\n", sep = "")}
generate_dna(9)
```

```
>len9
ACCTGGGGA
```

Q3. Write a generate_protein() function

```
# This function generates a random sequence of n amino acids
# with replacement. The function paste0 collapses the amino acid
# sequence into one string.
generate_protein <- function(n) {
  paste0(sample(c("A","R","N","D","C","E","Q","G","H","I",
```

```

        "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V"),
        n, replace = TRUE),
        collapse = "")
}
generate_protein(6)

```

```
[1] "EHAPWW"
```

Q4. Generate random protein sequences of length 6 to 13

```

# This function loops through the sequence lengths 6 to 13 and
# generates a random protein sequence for each length.
# Each sequence is printed in FASTA format with the 'cat()' function.
for (i in 6:13) {
  cat(">id.", i, "\n", generate_protein(i), "\n", sep = "")
}

```

```

>id.6
QDHEQQ
>id.7
FVVGRNK
>id.8
AHEYRTEL
>id.9
PKSYDHGCA
>id.10
CYTELDNPPD
>id.11
YKDCFWQIWPV
>id.12
AGKALRPLEMSP
>id.13
MRNDANSMWPVQI

```

Q5. BLASTp search against nr

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100.00%	100.00%	N
7	100.00%	100.00%	N
8	87.50%	100.00%	Y

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
9	88.89%	100.00%	Y
10	76.92%	100.00%	Y
11	90.00%	91%	Y
12	81.82%	92%	Y
13	100.00%	77%	Y

5a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?

From the table, the peptides started to look unique at length 8.

5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe.

The number of peptide combinations is 20^L , so as the length increases the number of combinations grow exponentially which makes it more probable to find a unique protein as the length of the peptide increases. The protein universe is extremely large, but isn't infinite so when the length of a peptide is short it can likely find a match by chance but as it increases it becomes less likely.

Q6. Connecting your findings to immunology (MHC class II and T-cell activation)

From the data in Q5, I would say the minimum length is 8-10 amino acids because this is the point where uniqueness occurred while still maintaining 100% coverage. Using very short peptides is a bad design choice as it's much more likely that these short sequences are to occur by chance in both the pathogen and the host. If the immune system used short protein sequences it can get confused and perform the wrong function through type I and type II errors. So by using a longer protein sequence it ensures that the sequence is unique enough to where errors won't occur.